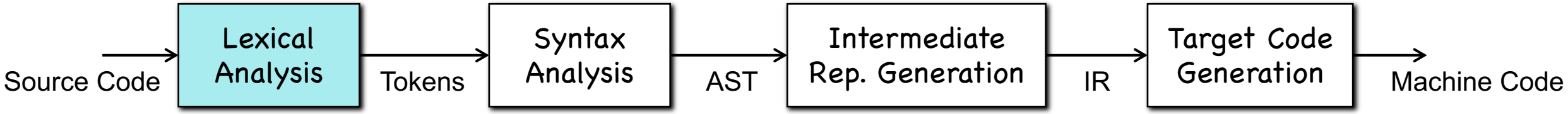
Made by Qingkai Shi
qingkaishi@nju.edu.cn
南京大学编译技术
及软件安全研究组

Chapter 3-1
Finite Automata

Lexical Analysis



- Find **lexemes** according to **patterns**, and create **tokens**
  - Lexeme – a character string
  - Pattern – regular expression (lexical errors if no patterns matched)
  - Token – <token-class-name, attribute>

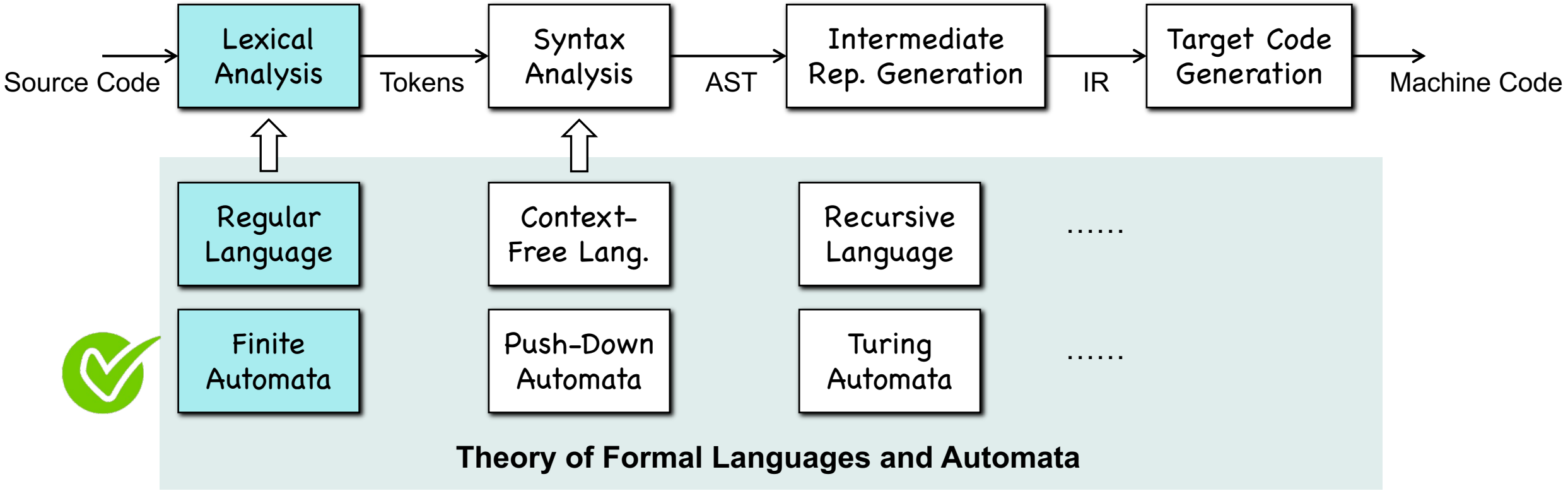
key	name	…
1	position	…
2	initial	…
…		…

symbol table

```
position  =  initial  +  rate  *  60
```

```
<id,1>  <=>  <id,2>  <+>  <id,3>  <*>  <number,60>
```

Lexical Analysis



PART I: Math Preliminaries

Alphabet and Strings

- A **language** is a set of strings, e.g., { cat, dog, … }
- A **string** is a sequence of letters defined over an **alphabet**
  - string example: cat, dog, …
  - alphabet example: **Σ** = { a, b, c, d, …, z }

- **Example**: consider a small alphabet **Σ** = { a, b }
  - strings: a, b, ab, aab, aabb, …
  - a language is any subset of { a, b, ab, aab, aabb, … }

String Operations

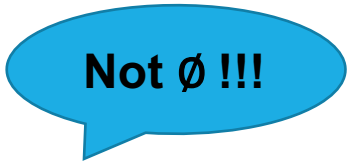
- **Concatenation**
  - $w_1 = aabb; w_2 = bbaa;$ → $w_1 w_2 = aabbbbaa;$
- **Reverse**
  - $w = a_1 a_2 a_3 a_4;$ → $w^R = a_4 a_3 a_2 a_1;$
- **Length**
  - $w = a_1 a_2 a_3 a_4;$ → $|w| = 4;$

Empty String and Sub-String

- **Empty String**: ϵ
  - $|\epsilon| = 0$
  - $\epsilon aabb = aab\epsilon\epsilon b = aabb\epsilon = aabb$
- **Substring**: a subsequence of consecutive characters
  - $abbab, abbab, abbab, abbab$
- **Prefix and Suffix**: $w = uv$, where u is prefix and v is suffix
  - prefix of abb includes: $\epsilon,\ a,\ ab,\ abb$
  - suffix of abb includes: $abb,\ bb,\ b,\ \epsilon$

Power, Kleene Star, and Plus

- **Power**: $w^n = \underbrace{ww \ldots w}_{n}$; $w^0 = \epsilon$;
  - $(abb)^2 = abbabb$
  - $(abb)^0 = \epsilon$
- **Kleene Star**: $\boldsymbol{\Sigma} = \{ a, b \} \Rightarrow \boldsymbol{\Sigma}^* = \{ \epsilon, a, b, ab, abb, aab, \ldots \} \ldots$
- **Plus**: $\boldsymbol{\Sigma}^+ = \boldsymbol{\Sigma}^* - \{\epsilon\}$



- **Language:** a language is any subset of $\boldsymbol{\Sigma}^*$, e.g., $\{\epsilon\}$, $\{\epsilon, a, b\}$, …

Operations on Languages

- Usual set operations
  - $L_1 = \{a, b\}; L_2 = \{a, ab\}; \Rightarrow L_1 \cup L_2 = \{a, b, ab\}$
  - $L_1 = \{a, b\}; L_2 = \{a, ab\}; \Rightarrow L_1 \cap L_2 = \{a\}$
  - $L_1 = \{a, b\}; L_2 = \{a, ab\}; \Rightarrow L_1 - L_2 = \{b\}$

- Complement: $\bar{L} = \Sigma^* - L$
- Reverse: $L^R = \{w^R : w \in L\}$
- Concatenation: $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$

Operations on Languages

- **Power**: $L^n = \underbrace{LL \ldots \ldots L}_{n};\qquad L^0 = \{\epsilon\}$

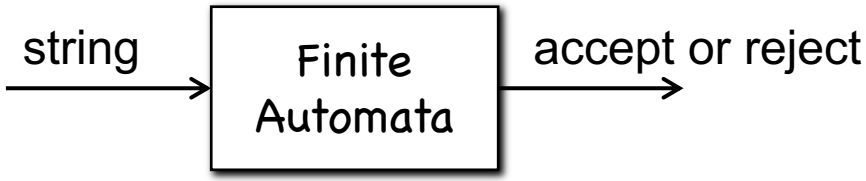
- **Star-Closure**: $L^* = L^0 \cup L^1 \cup L^2 \cup \cdots$
- **Positive-Closure**: $L^+ = L^1 \cup L^2 \cup \cdots = L^* - \{\epsilon\}$

- **Quiz 1**: $L = \{a, b\}\,; L^3 = ?$
- **Quiz 2**: $L = \{a^n b^n : n \geq 0\}; L^2 = ?$

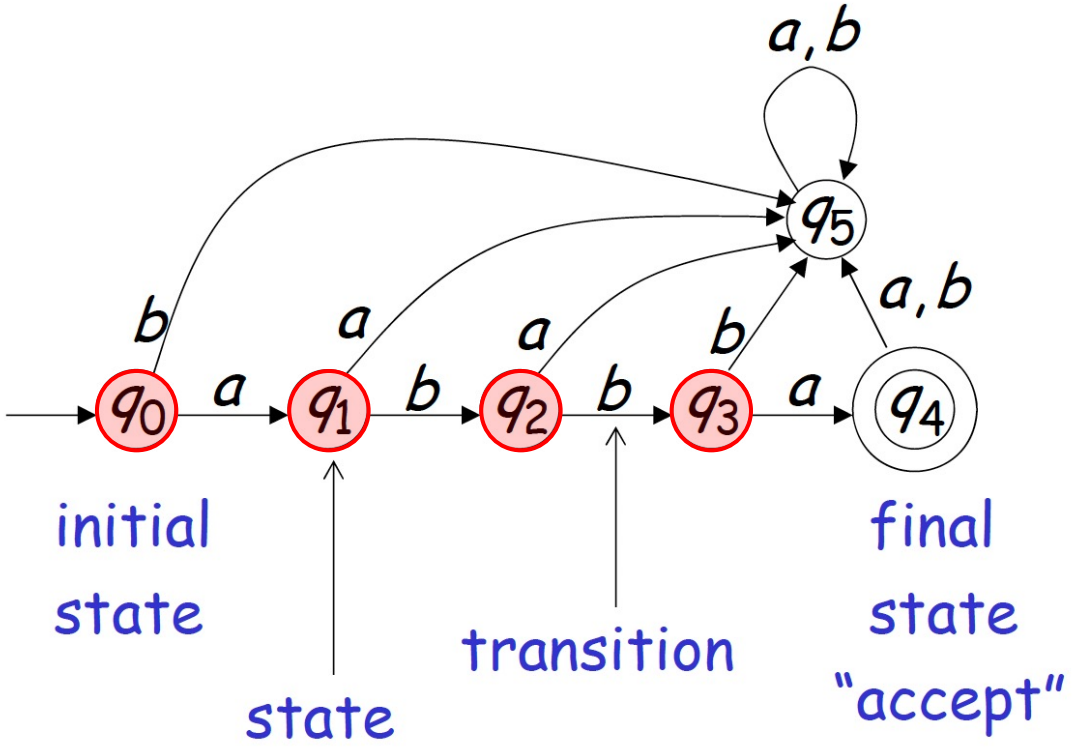
PART II:
Deterministic Finite Automata

Finite Automata

- Input a string, output “accept” or “reject”



- **Example**: finite automata for *abba*
- *What if we input “abb”?*
- **Deterministic** Finite Automata!



Deterministic Finite Automata

- A DFA is a five-tuple: $(Q, \Sigma, \delta, q_0, F)$
  - Q: A finite set of states
  - Σ: A finite set of input characters, i.e., an alphabet
  - δ: $Q \times \Sigma \longmapsto$ Q: Transition function, e.g., $\delta(q, a) = q'$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

- An extension of transition: $\delta^*: Q \times \Sigma^* \longmapsto$ Q
  - $\delta^*(q, abba) = q'; \quad \delta^*(q, \epsilon) = q;$

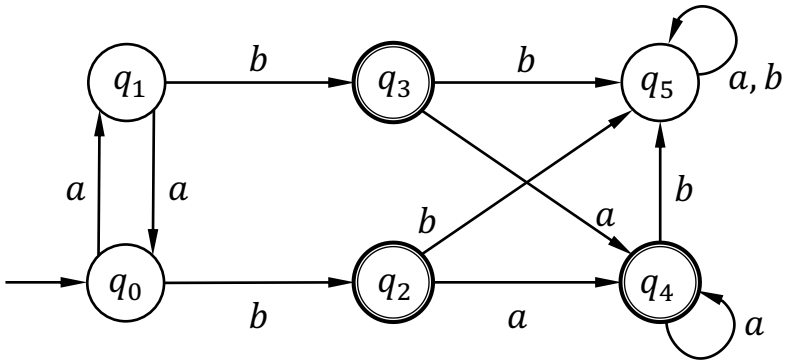
Defining a Language by DFA

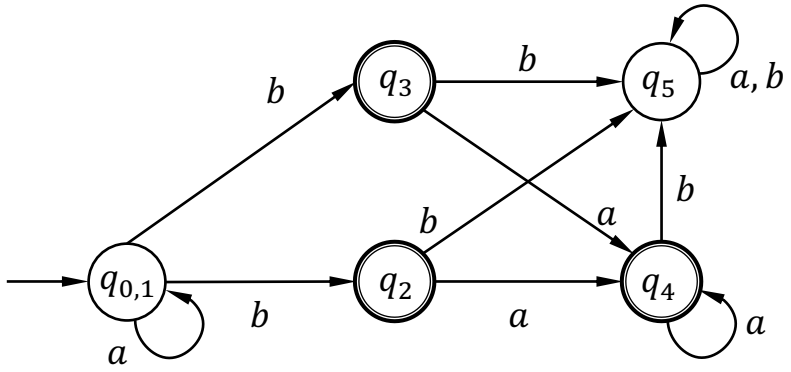
- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \subseteq F\}$
- **Common application**: checking if a string $w \in L(M)$

```
void check(w, M) {
    q = q₀;
    while (true) {
        c = read(w); if (c == None) { print(q ∈ F ? “accept” : “reject”); }
        switch(q) {
        case q₀: if (c == ‘a’) { q = q₁; } break; // δ(q₀,a)=q₁; δ(q₀,!a)=q₀
        case q₁: … break;
        case q₂: … break;
        case ……
        }
    }}
```

DFA Minimization

- Minimizing DFA can improve the efficiency of computation

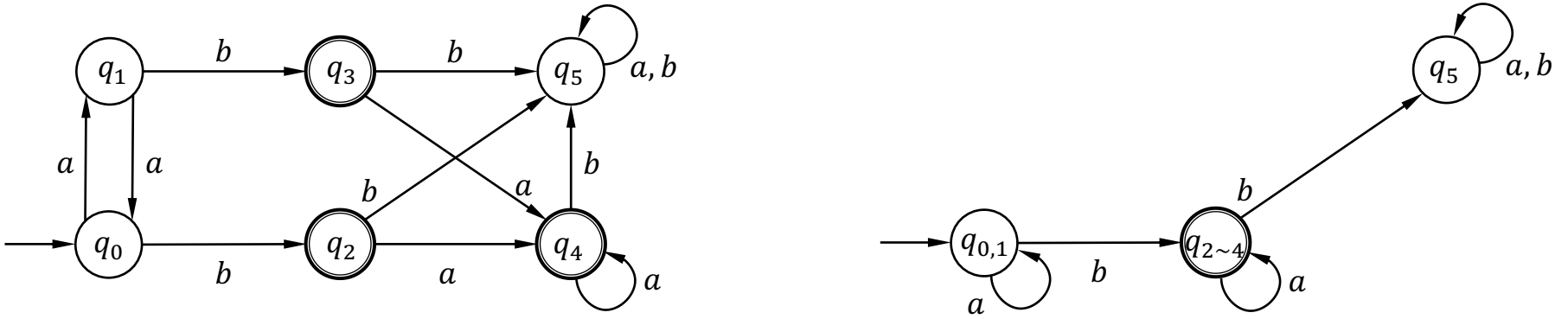




Step 1	$\{q_0, q_1, q_5\}, \{q_2, q_3, q_4\}$	Distinguish final and non-final states
Step 2	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	$\delta(q_0, a/b)$ and $\delta(q_1, a/b)$ are in the same set, but $\delta(q_5, a/b)$ not
Step 3	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	The result does not change and the algorithm completes

DFA Minimization

- Minimizing DFA can improve the efficiency of computation



- **We remove all unreachable states before the above steps**

DFA Minimization

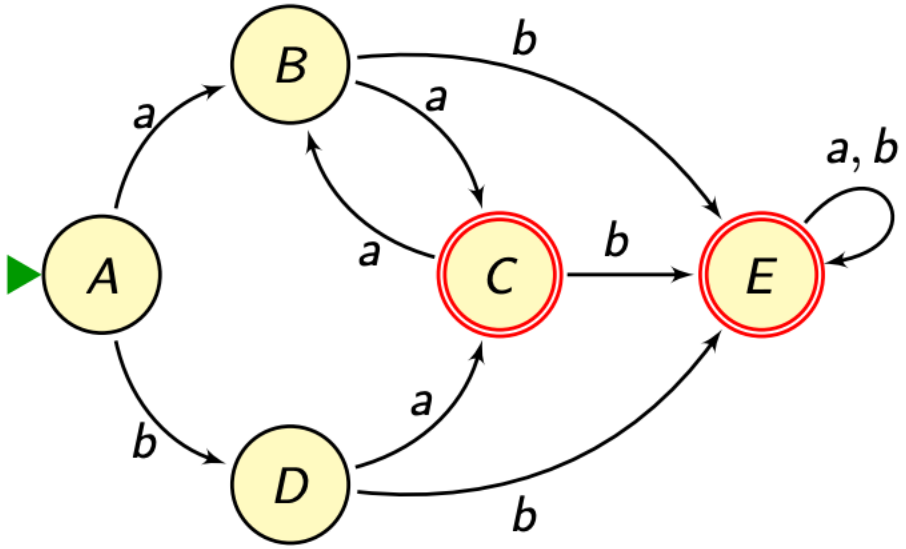
- Best Average Complexity: $O(n \log \log n)$!

Theoretical Computer Science
Volume 417, 3 February 2012, Pages 50-65

Average complexity of Moore's and Hopcroft's algorithms

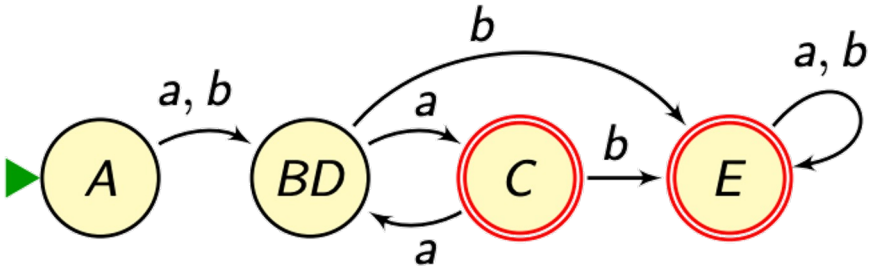
Julien David

DFA Minimization



Have a Try!!

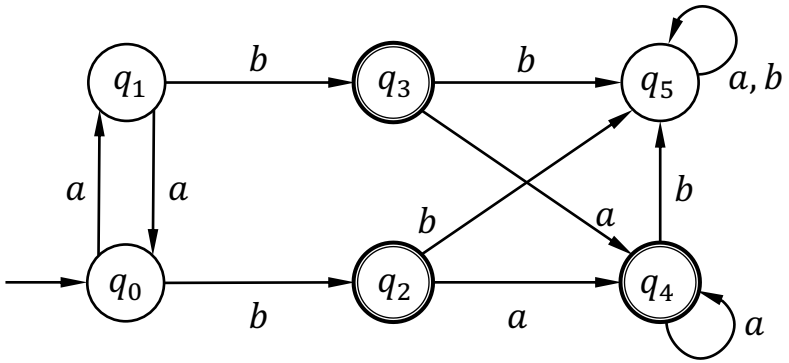
DFA Minimization

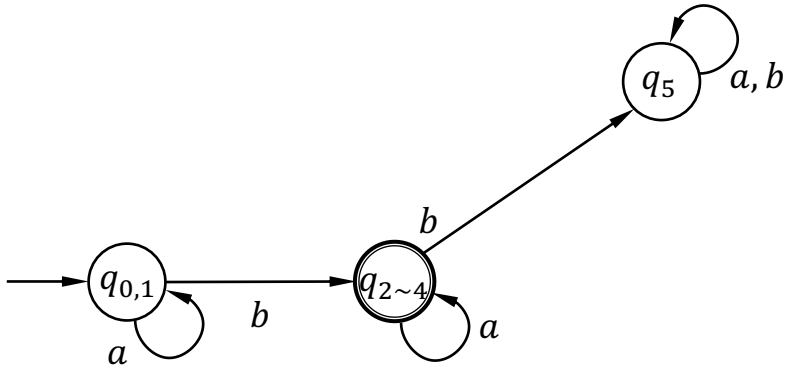


Solution

DFA Bi-Simulation

- Checking the equivalence of the DFAs, i.e., $L(M_1) = L(M_2)$



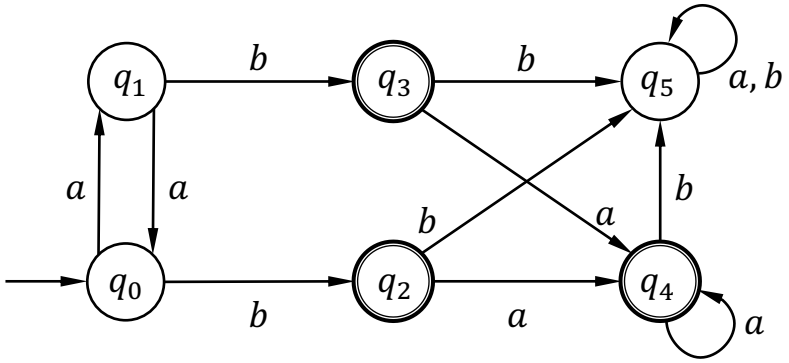


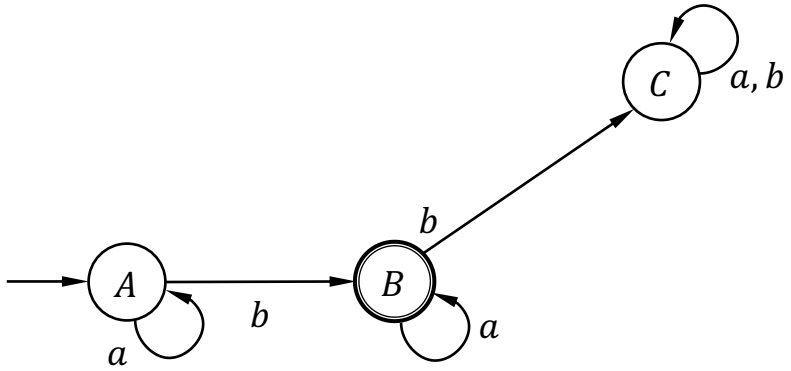
DFA Bi-Simulation

- Checking the equivalence of the DFAs, i.e., $L(M_1) = L(M_2)$
  - $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \subseteq F\}$
- Given an input string, let M_1 and M_2 evaluate it at the same time, M_1 reaches a final state if and only if M_2 reaches a final state

DFA Bi-Simulation

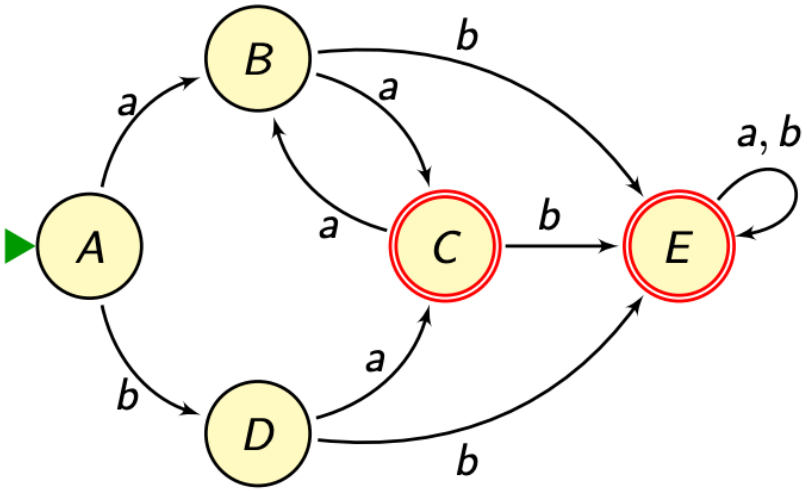
- Checking the equivalence of the DFAs, i.e., $L(M_1) = L(M_2)$

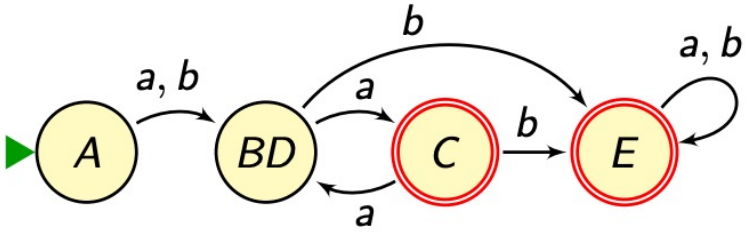




State Pairs	a	b	
$\{ q_0, A \}$	$\{ q_1, A \}$	$\{ q_2, B \}$	
$\{ q_1, A \}$	$\{ q_0, A \}$	$\{ q_3, B \}$	
$\{ q_2, B \}$	$\{ q_4, B \}$	$\{ q_5, C \}$	M_1 reaches final state if and only if M_2 reaches final state
$\{ q_3, B \}$	$\{ q_4, B \}$	$\{ q_5, C \}$	
$\{ q_4, B \}$	$\{ q_4, B \}$	$\{ q_5, C \}$	
$\{ q_5, C \}$	$\{ q_5, C \}$	$\{ q_5, C \}$	

DFA Bi-Simulation



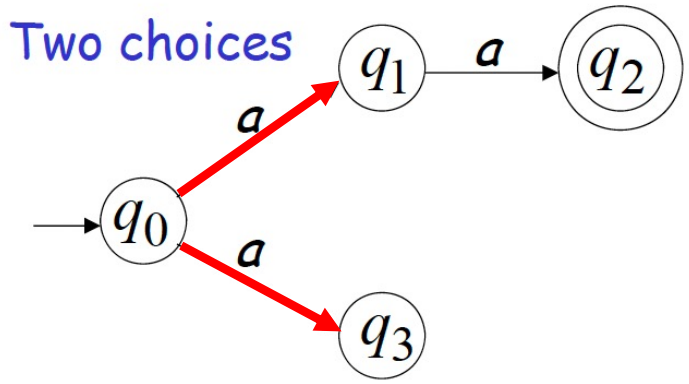


State Pairs	a	b	
(A, A')	(B, BD')	(D, BD')	
(B, BD')	(C, C')	(E, E')	
(D, BD')	(C, C')	(E, E')	M_1 reaches final state if and only if M_2 reaches final state
(C, C')	(B, BD')	(E, E')	
(E, E')	(E, E')	(E, E')	

PART III:
Non-deterministic Finite Automata

Non-deterministic Finite Automata

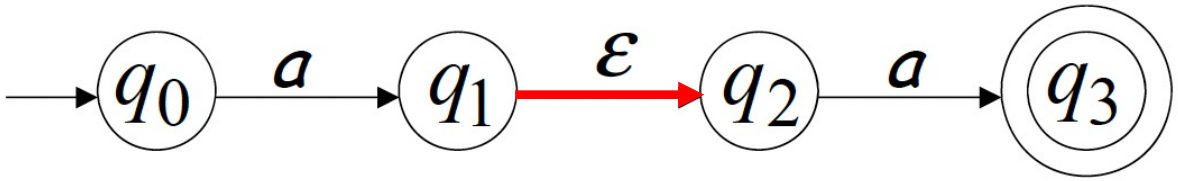
- There are multiple choices of state transition



- Given a string, *aa*, there are two choices for the first transition
- A string, e.g., *aa*, can be accepted by NFA as long as there exists one computation of the NFA accepts the string

ϵ-Transition

- Transition to the next states without reading any inputs



- NFA also allows ϵ-transitions!

Definition of NFA

- A ~~DFA~~NFA is a five-tuple: $(Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - Q: A finite set of states
  - Σ: A finite set of input characters, i.e., an alphabet
  - δ: $Q \times (\Sigma \cup \{\epsilon\}) \longmapsto$ ~~Q~~2^Q: Transition function, e.g., $\delta(q, a) = \{q', q''\}$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

ϵ-Closure

- ϵ-*closure(q)* returns all states *q* can reach via ϵ-transitions, including *q* itself

- An extension of transition: $\delta^*: Q \times \Sigma^* \longmapsto 2^Q$
  - $q' \in \delta^*(q, w)$, where $w \in \Sigma^*$, if and only if
    - (1) there's a walk from q to q'' with w
    - (2) $q' \in \epsilon\text{-}closure(q'')$

Defining a Language by NFA

- **Recap: language defined by DFA**
- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \subseteq F\}$
- Take a NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$, language can be accepted by the NFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}$

NFA = DFA

- Every DFA is trivially an equivalent NFA
- Every NFA N can be converted to an equivalent DFA D
  - Every string accepted by the NFA is accepted by the DFA
  - Every string rejected by the NFA is rejected by the DFA
  - i.e., $L(N) = L(D)$

From NFA to DFA

- Given an NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - **Step 1:** initialize a DFA with a start state $\{ q_0 \}$, which is a set of NFA states
  - **Step 2:** for each DFA state $\{ q_i, q_j, \ldots, q_m \}$, and char in the alphabet, $a \in \Sigma$

$$\text{let } S = \bigcup \begin{cases} \delta^*(q_i, a \in \Sigma) \\ \delta^*(q_j, a \in \Sigma) \\ \ldots \\ \delta^*(q_m, a \in \Sigma) \end{cases}$$

if any q_i is a final state, S is a final state of the DFA

  - **Step 3**: add transition $\delta(\{ q_i, q_j, \ldots, q_m \}, a) = S$ in the DFA

From NFA to DFA

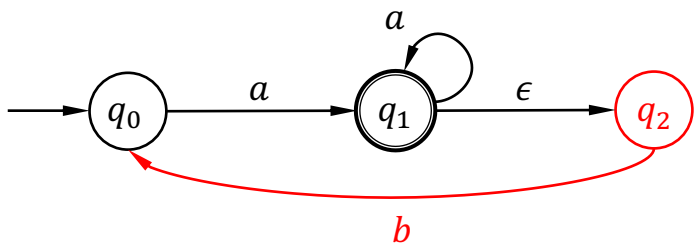
- Given an NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - **Step 1:** initialize a DFA with a start state $\{ q_0 \}$, which is a set of NFA states
  - **Step 2:** for each DFA state $\{ q_i, q_j, \ldots, q_m \}$, and char in the alphabet, $a \in \Sigma$

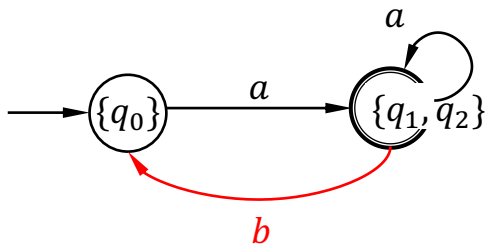
$$\text{let } S = \bigcup \begin{cases} \delta^*(q_i, a \in \Sigma) \\ \delta^*(q_j, a \in \Sigma) \\ \ldots \\ \delta^*(q_m, a \in \Sigma) \end{cases}$$

For any input string, the DFA and the NFA reach the final state at the same time, thus DFA=NFA

  - **Step 3**: add transition $\delta(\{ q_i, q_j, \ldots, q_m \}, a) = S$ in the DFA

From NFA to DFA

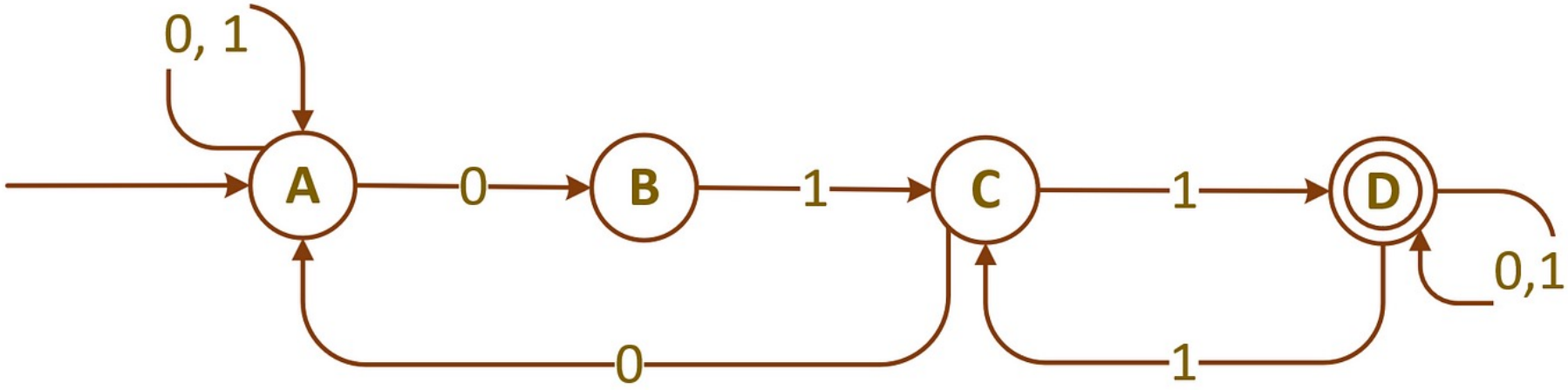




Because the DFA states consist of sets of NFA states, an *n*-state NFA may be converted to a DFA with at most 2^n states. For every *n*, there exist *n*-state NFAs such that every subset of states is reachable from the initial subset, so that the converted DFA has exactly 2^n states, giving $\Theta(2^n)$ worst-case time complexity.

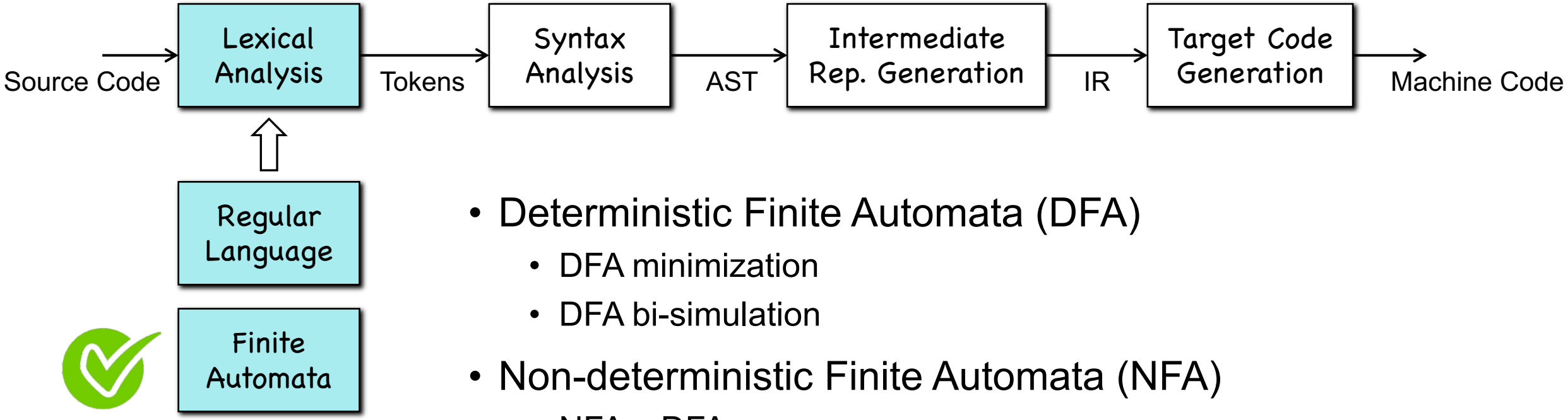
When converting an NFA to a DFA, there is no guarantee that we will have a smaller DFA.

From NFA to DFA



Have a Try!!

Summary

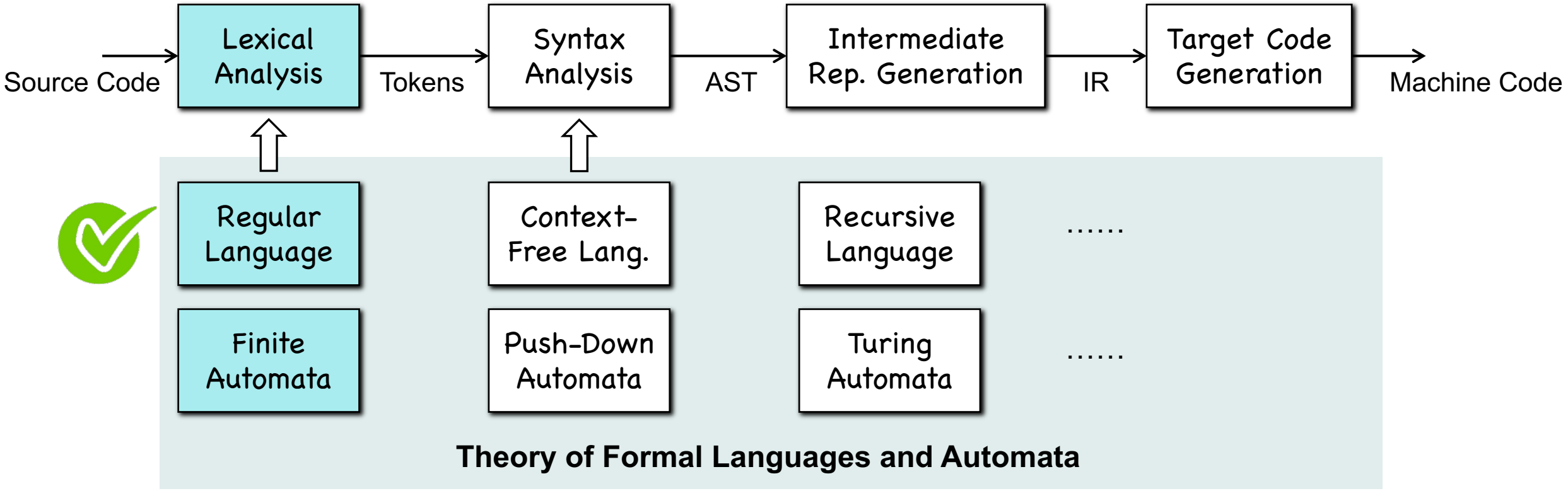


- Deterministic Finite Automata (DFA)
  - DFA minimization
  - DFA bi-simulation
- Non-deterministic Finite Automata (NFA)
  - NFA = DFA
  - NFA → DFA
- Languages defined by DFA/NFA → Regular Language

Made by Qingkai Shi
qingkaishi@nju.edu.cn
南京大学编译技术
及软件安全研究组

Chapter 3-2
Lexical Analysis

Lexical Analysis



PART I: Regular Language

Regular Language

- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \subseteq F\}$
- A language L is regular if there is a DFA M such that $L = L(M)$
- **Examples**: $\{abba\}$ $\{\varepsilon, ab, abba\}$ $\{a^n b : n \geq 0\}$

  { all strings with prefix ab }

  { all strings with suffix ab }

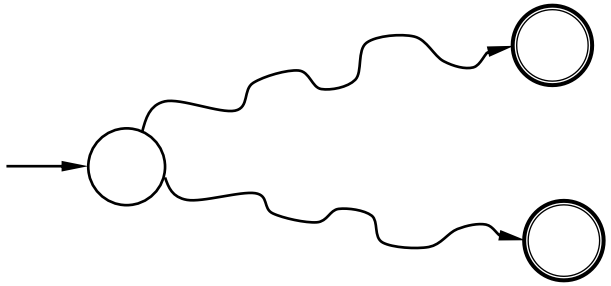
Regular Language

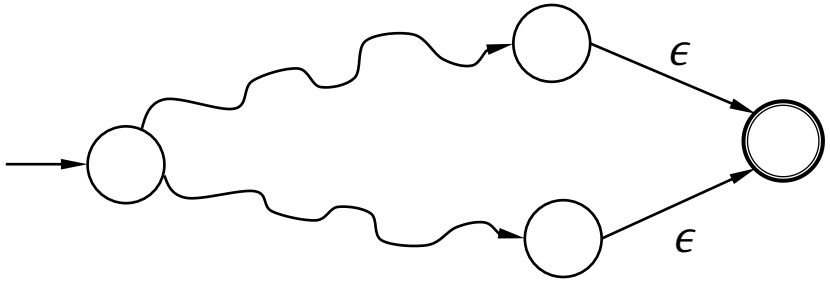
- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \subseteq F\}$
- Take a NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$, language can be accepted by the NFA is written as $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}$

L is a regular language if there's a DFA/NFA M such that $L = L(M)$

Closure Properties

- Any NFA can be converted into an NFA with a single final state

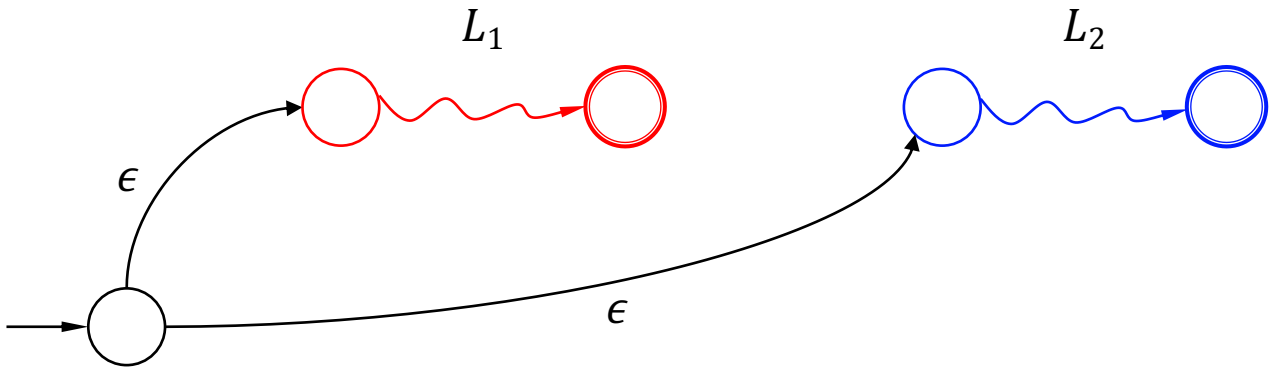




- Every regular language has an equivalent and single-exit NFA

Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$

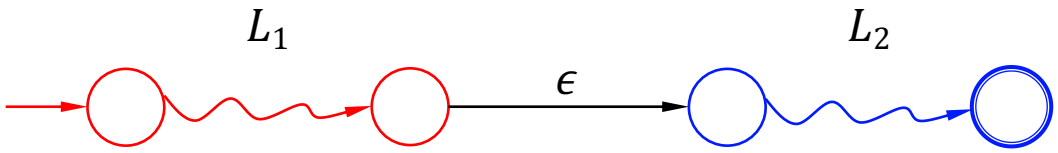


Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$

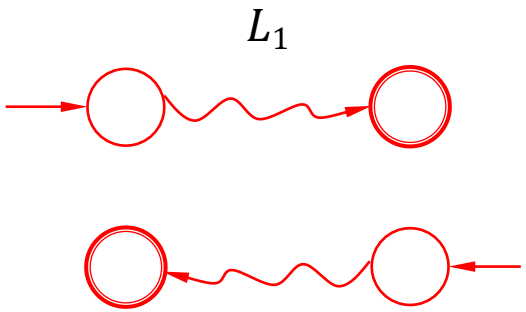
Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



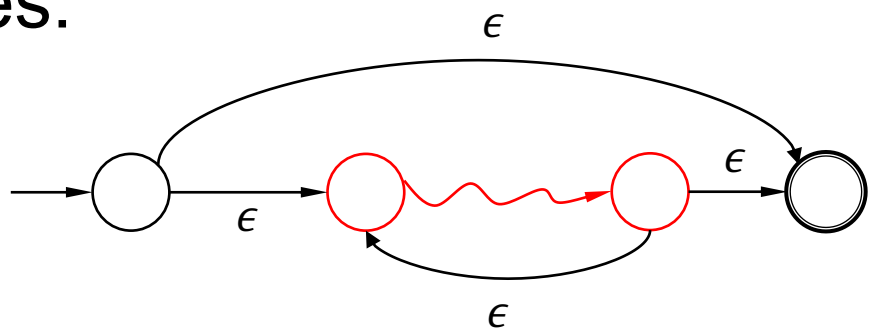
Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



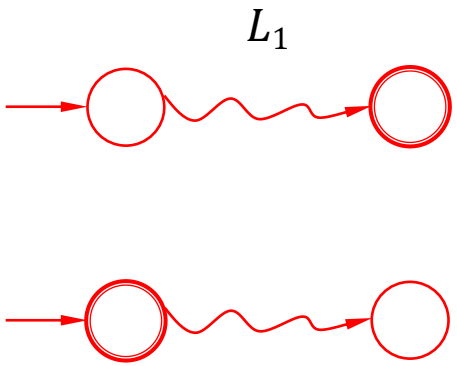
Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$

Regular language is closed under union, intersection, reversal, complement, concatenation, and Kleene star.

PART II: Regular Expression

Regular Expression (Regex)

- Regex describes a language
- Primitive regex
  - $\emptyset, \epsilon, a$
- Given two regex: r_1, r_2, the following are regex
  - $r_1|r_2$
  - $r_1 r_2$
  - r_1^*
  - (r_1)
- **Example:** $(a|b)^*c$

Language by Regex

- The language represented by regex are defined below
- Primitive regex
  - $L(\emptyset) = \emptyset$; $L(\epsilon) = \{\epsilon\}$; $L(a) = \{a\}$
- Given two regex: r_1, r_2, the following are regex
  - $L(r_1|r_2) = L(r_1) \cup L(r_2)$
  - $L(r_1 r_2) = L(r_1)L(r_2)$
  - $L(r_1^*) = (L(r_1))^*$
  - $L((r_1)) = L(r_1)$
- Two regex are equivalent if they represent the same language

Laws of Regex

- Commutativity and Associativity
  - $r_1|r_2 = r_2|r_1$
  - $(r_1|r_2)|r_3 = r_1|(r_2|r_3)$
  - $(r_1 r_2)r_3 = r_1(r_2 r_3)$
- Identities and Annihilators
  - $r_1|\emptyset = \emptyset|r_1 = r_1$
  - $r_1\epsilon = \epsilon r_1 = r_1$
  - $r_1\emptyset = \emptyset r_1 = r_1$

Laws of Regex

- Distributive and Idempotent Law
  - $(r_1|r_2)\,r_3 = r_1r_3|r_2r_3$
  - $r_3(r_1|r_2) = r_3r_1|r_3r_2$
  - $r_1|r_1 = r_1$
- Closure
  - $(r_1^*)^* = r_1^*$
  - $\emptyset^* = \epsilon^* = \epsilon$
  - $r_1^+ = r_1(r_1^*) = (r_1^*)r_1$
  - $r_1? = \epsilon|r_1$

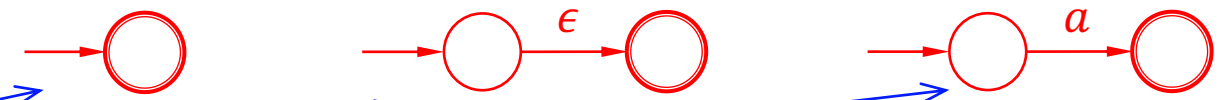
Regex = Regular Language

- (1) Any regex represents a **regular** language
- (2) Any regular language can be expressed by a regex

Regex to NFA (DFA)

- (1) Any regex represents a ~~regular language~~NFA/DFA

Regex to NFA (DFA)

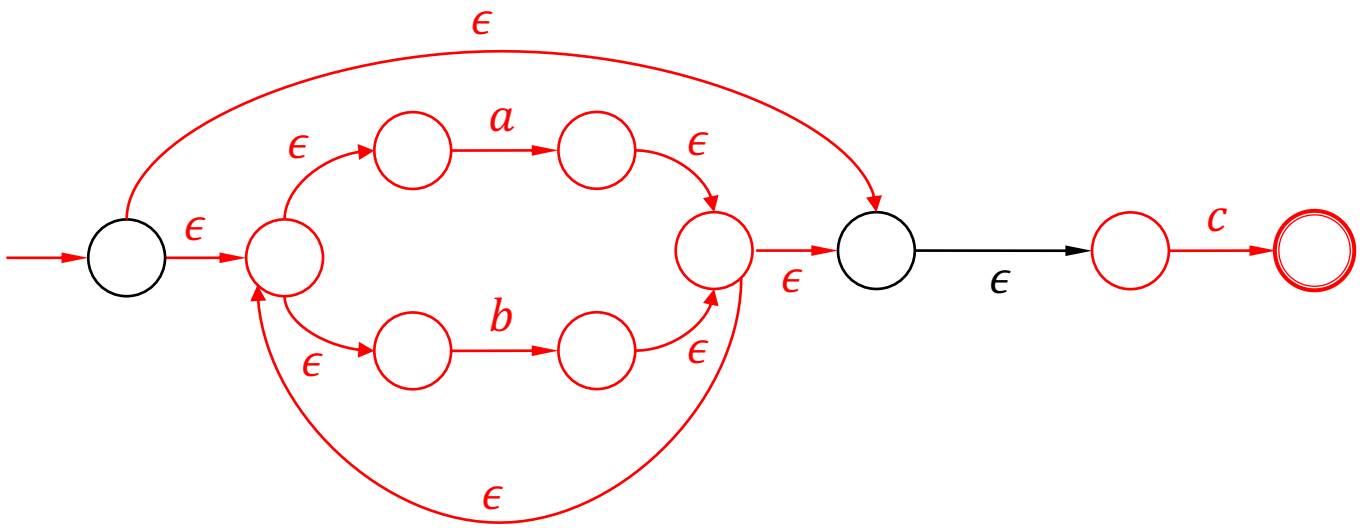


- Primitive regex
  - $L(\emptyset) = \emptyset;\ \ L(\epsilon) = \{\epsilon\};\ \ L(a) = \{a\}$
- Given two regex: r_1, r_2, the following are regex
  - $L(r_1|r_2) = L(r_1) \cup L(r_2)$
  - $L(r_1 r_2) = L(r_1)L(r_2)$
  - $L(r_1^*) = (L(r_1))^*$
  - $L((r_1)) = L(r_1)$

- *Recall how we prove regular language closed under union, concatenation, and Kleene star*

Example

- Build the NFA for the regex: $(a|b)^*c$
- $L\big((a|b)^*c\big) = \big(L(a|b)\big)^* L(c) = \big(L(a) \cup L(b)\big)^* L(c)$

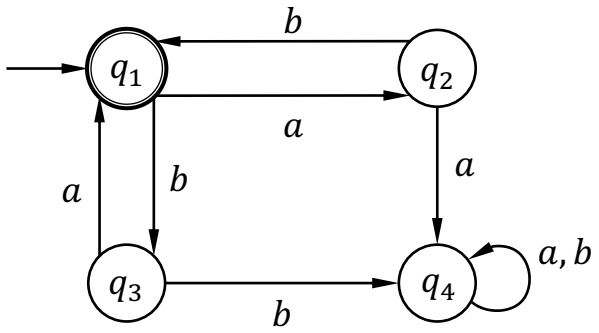


DFA to Regex

- (2) Any ~~regular language~~DFA can be expressed by a regex

DFA to Regex

- (2) Any ~~regular language~~DFA can be expressed by a regex



$$q_1 = \epsilon \mid q_2 b \mid q_3 a$$

$$q_2 = q_1 a$$

$$q_3 = q_1 b$$

$$q_4 = q_2 a \mid q_3 b \mid q_4 a \mid q_4 b$$

Variable Elimination

$$q_1 = \epsilon \mid q_1 ab \mid q_1 ba$$

Associativity

$$q_1 = \epsilon \mid q_1 (ab|ba)$$

Arden's Theorem:
R = QP is the solution of R = Q | RP*

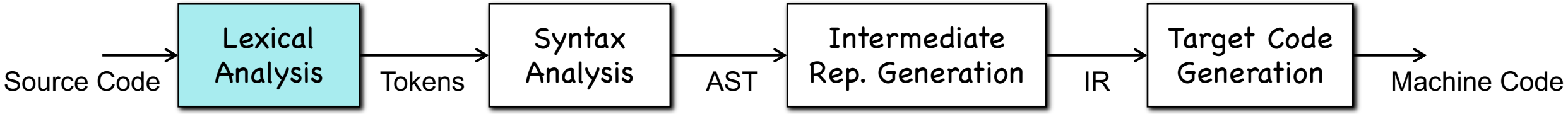
$$q_1 = \epsilon\ (ab|ba)* = (ab|ba)*$$

Elementary Questions

- Given a regular language L by a regex, check if a string $w \in L$
- Build a DFA and check if the DFA accepts the string
- This is the foundation of the compilers' lexical analysis
  - **Step 1**: Write regex for lexemes (e.g., numbers, variables), build DFAs
  - **Step 2**: Regard the source code as an input string
  - **Step 3**: Check the input string against the DFAs to recognize lexemes

PART III: Lexical Analysis

Lexical Analysis



- Find **lexemes** according to **patterns**, and create **tokens**
  - Lexeme – a character string
  - Pattern – regular expression (lexical errors if no patterns matched)
  - Token – <token-class-name, attribute>

Regexp → NFA → DFA → min DFA

key	name	…
1	position	…
2	initial	…
…		…

symbol table

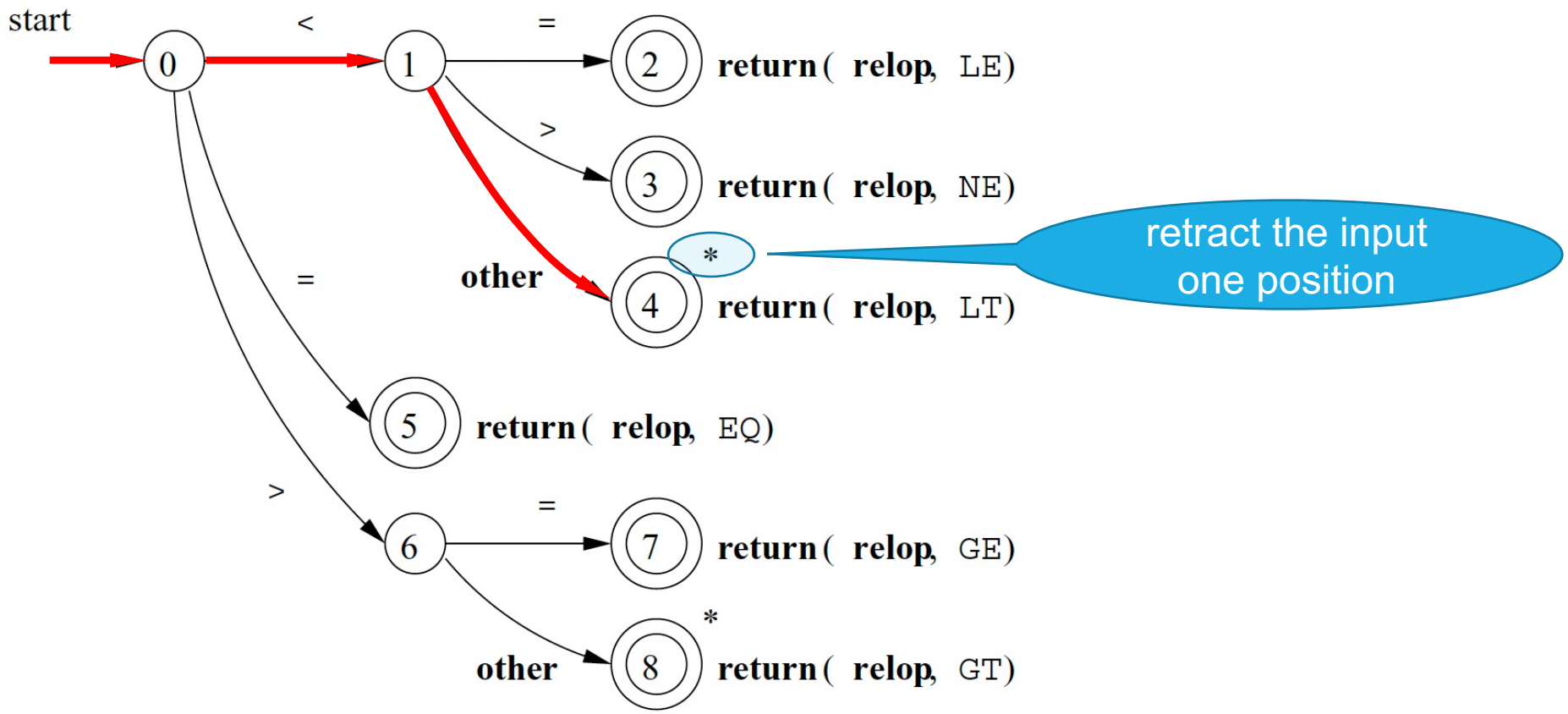
```
position  =  initial  +  rate  *  60
```

```
<id,1>  <=>  <id,2>  <+>  <id,3>  <*>  <number,60>
```

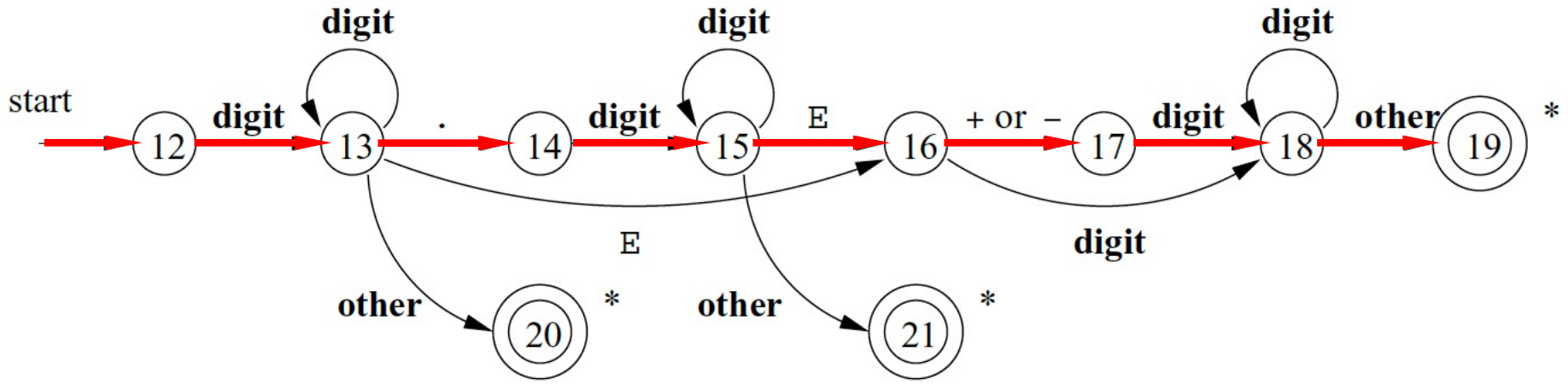
Lexical Analysis

- Input the source code (as an input string) into a lexical analyzer
- Check against the DFAs of keywords/operators/identifiers/…
- Whenever reaching a final states of a DFA, create a token

Example: Relational Operator



Example: Unsigned Numbers



Example: Identifiers

- Try to write the regex of identifiers for a C-like language and build the NFA

What if Multiple DFAs Work

- Priority of DFAs
- Matching the longest string
- …

PART IV: Pumping Lemma

Question

- Is the language represented by $a^n b^m$ regular?
- Yes!
- Regex: $a^* b^*$

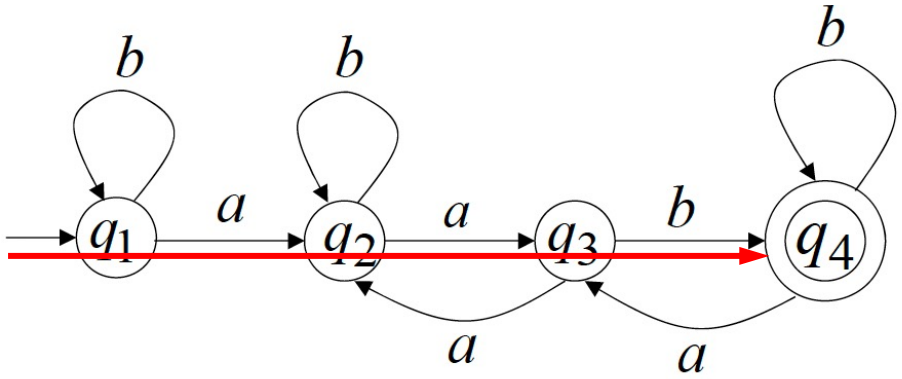
Question

- Is the language represented by $a^n b^n$ regular?
- No!
- Why? Prove it!

Observation

- For any DFA and string w, $|w| \geq$ # states,
- There must be a state q that repeats in the walk of w

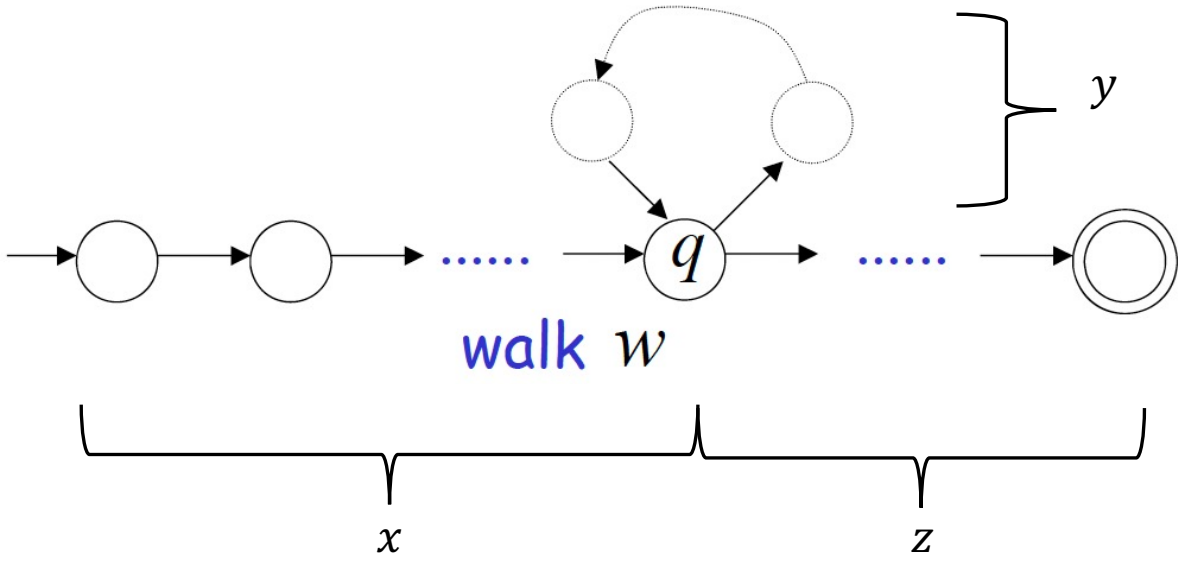
- Example:



A walk without repeating any state yields a string of length ≤ 3

Observation

- $|w| \geq m$ (assume m is the number of states)
- $w_i = xy^iz$ where $|xy| \leq m; |y| \geq 1$
- For any i, w_i is in the language of the automata



The Pumping Lemma

- If L is an infinite regular language (the set L is infinite)
- there must exist an integer m
- for any string $w \in L$ with length $|w| \geq m$
- we can write $w = xyz$ with $|xy| \leq m$ and $|y| \geq 1$
- such that $w_i = xy^iz \in L$

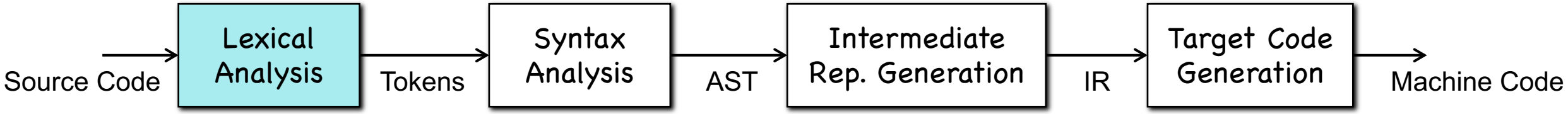
Contrapositive Proportion

- For any integer m
- if there exists $w \in L$ such that $|w| \geq m$
- choose x, y, and z such that $w = xyz$, $y \neq \epsilon$, and $|xy| \leq m$
- if there exists k, such that $xy^k z \notin L$
- ➔
- L is not regular

Proving $L = \{a^n b^n\}$ is not Regular

- For any integer m Let m = 3
- if there exists $w \in L$ such that $|w| \geq m$ Let w = aaabbb
- choose x, y, and z such that $w = xyz$, $y \neq \epsilon$, and $|xy| \leq m$ Let w = xyz, where x = a; y = a; z = abbb
- if there exists k, such that $xy^k z \notin L$ Let k = 2, $xy^kz = a^4b^3 \notin L$
- ➔
- L is not regular a^nb^n is not regular

Summary



- Find **lexemes** according to **patterns**, and create **tokens**
  - Lexeme – a character string
  - Pattern – regular expression (lexical errors if no patterns matched)
  - Token – <token-class-name, attribute>

Regexp → NFA → DFA → min DFA

key	name	…
1	position	…
2	initial	…
…		…

symbol table

```
position  =  initial  +  rate  *  60
```

```
<id,1>  <=>  <id,2>  <+>  <id,3>  <*>  <number,60>
```